

BikerLink

AI DB Integrity — Report di Diagnosi

Data: 4 giugno 2026 • Ambiente: Sviluppo • Check eseguiti: 64

' Nessun problema reale rilevato

Gli 8 allarmi "high" visibili nel pannello erano falsi positivi causati da un bug nel codice del check. Il bug è stato identificato, corretto e le violazioni chiuse. I dati del matching sono integri.

1. Riepilogo Esecutivo

Il pannello "AI DB Integrity" mostrava 8 violazioni con severity "high" raggruppate in due check distinti (orphans/bb-matches-b1::broken e orphans/bb-matches-b2::broken).

Dopo un'analisi approfondita del codice sorgente e del database, è emerso che questi allarmi erano interamente causati da un bug nella funzione di sanitizzazione dei nomi di colonna SQL: la regex `[^a-z_]` rimuoveva i caratteri numerici, trasformando i nomi di colonna corretti `"biker1_id"` e `"biker2_id"` in `"biker_id"` (inesistente), facendo fallire la query SQL.

I dati reali del matching (tabella `biker_biker_matches`) sono integri: 0 orfani su entrambe le colonne. Il sistema di matching non ha problemi nei dati — solo il check di controllo era rotto.

2. Stato Violazioni — Prima e Dopo il Fix

Check ID	Severity	Tipo	Stato Prima	Stato Dopo
orphans/bb-matches-b1	HIGH	::broken (bug)	manual_pending ×4	ignored '
orphans/bb-matches-b2	HIGH	::broken (bug)	manual_pending ×4	ignored '
duplicates/tags-same-slug	LOW	Dato reale	manual_pending ×4	manual_pending



3. Root Cause Analysis — Il Bug

Dove:
server/ai/db-integrity/checks/
orphans.ts (funzioni
orphanCheck e

deleteOrphans)

Perché:

La sanitizzazione dei nomi di colonna usava una regex che escludeva le cifre.

PRIMA (bug):

```
const safeFk = fk.replace(/^[a-z_]/g, "");  
// "biker1_id" !' "biker_id" (il "1" viene eliminato)  
// "biker2_id" !' "biker_id" (il "2" viene eliminato)
```

DOPO (fix):

```
const safeFk = fk.replace(/^[a-z0-9_]/g, "");  
// "biker1_id" !' "biker1_id" '  
// "biker2_id" !' "biker2_id" '
```

La stessa regex era presente in tre punti:

1. orphanCheck() in orphans.ts — corretto
2. deleteOrphans() in orphans.ts — corretto
3. orphanCheck() locale in orphans-extra.ts — corretto

Il bug era introdotto dallo stesso scaffolding iniziale (Task #2536) per tutti e tre i punti.

4. Meccanismo di Propagazione (Effetto Cascata)

Il runner (runner.ts) ha un meccanismo di sicurezza: se un check lancia un'eccezione SQL, invece di mascherarla come "ok" (che permetterebbe al sistema di sembrare verde anche con check rotti), genera automaticamente una violazione sintetica con:

- check_id: <id-originale>::broken
- severity: high (forzato — un check rotto non può passare inosservato)
- status: manual_pending
- details: { broken: true, error: "<messaggio SQL>" }

Questo meccanismo è corretto e utile. Il problema è che la causa dell'eccezione era il bug nella regex, non un problema reale nei dati. Il check avrebbe dovuto passare con 0 orfani.

Il cron (ore 01:00 ogni notte) ha eseguito lo scan per 4 giorni consecutivi (1-4 giugno), generando 2 violazioni ::broken per run = 8 violazioni totali accumulate.

5. Stato Reale dei Dati — Matching

I dati nella tabella biker_biker_matches sono integri. Eseguendo le query con i nomi di colonna corretti (biker1_id, biker2_id), il risultato è:

Query verificata	Orfani trovati	Esito
biker_biker_matches.biker1_id != users.id	0	' OK — nessun orfano
biker_biker_matches.biker2_id != users.id	0	' OK — nessun orfano

La colonna "biker_id" che il check cercava di verificare non esiste: i nomi corretti sono "biker1_id" e "biker2_id" come definito nello schema della tabella.

Tutti gli altri check di integrità del matching (biker_zavorrina_matches, match_preferences, match_feedback, embeddings, sessions, etc.) sono passati senza violazioni.

6. Unica Violazione Reale Aperta — Bassa Priorità

È presente una sola violazione reale, con severity LOW, non urgente:

Check	Severity	Count	Rilevato da	Dettaglio
duplicates/tags-same-slug	LOW	1	4 run cron	Slug "tourer" associato a 2 tag diversi (IDs diversi)

IDs coinvolti: a24345d0-a03b-4a6e-b971-f7d62d7987da e 330a22ab-ef92-49eb-9a15-3c2b8f0ad907
Azione consigliata: verificare manualmente quale dei due record è corretto e rimuovere il duplicato.
Non impatta funzionalità: il sistema seleziona un tag a caso quando ci sono slug duplicati.

7. Azioni Eseguite

#		Azione	Note
1	'	Fix regex orphanCheck() in orphans.ts	[^a-z_]' [^a-z0-9_] — preserva cifre nei nomi colonna
2	'	Fix regex deleteOrphans() in orphans.ts	Stessa correzione per la funzione di autofix
3	'	Fix regex orphanCheck() in orphans-extra.ts	Copia locale della funzione nello stesso file
4	'	Chiusura 8 violazioni false positive nel DB	Status: manual_pending != ignored (con timestamp)
5	'	Rebuild server backend	Checksum aggiornato, server healthy

8. Comportamento Atteso ai Prossimi Scan

Il prossimo scan cron (5 giugno 2026, ore 01:00) eseguirà i 64 check con il codice corretto. I check orphans/bb-matches-b1 e orphans/bb-matches-b2 passeranno con 0 orfani.

Risultato atteso dopo il fix:

- Nessuna violazione "high" o "critical"
- Solo la violazione "low" del tag duplicato "tourer" (se non risolta prima)
- Pannello: GIALLO (per il low) o VERDE (se il tag viene risolto)

Per forzare un run immediato è possibile usare il pulsante "Esegui ora" nel pannello admin !' Al DB Integrity senza aspettare il cron notturno.

9. Storico Run di Sistema

Data	Trigger	Check eseguiti	Violazioni	Auto-fix	Pending	Durata (ms)
2026-06-04	cron	64	3*	0	3*	573
2026-06-03	cron	64	3*	0	3*	273
2026-06-02	cron	64	3*	0	3*	145
2026-06-01	cron	64	3*	0	3*	345

** 2 violazioni "high" erano false positive (::broken bug), 1 "low" era reale.
Totale: 3 per run.*

